

Return Risk Intelligence: Predicting Return Fraud & Product Quality Issues

A PySpark/MLlib pipeline that triages e-commerce returns into fraud, policy abuse, and genuine quality issues — so audit capacity goes where it's actually needed.

THE PROBLEM

Manual return audits don't scale — and they treat three different problems as one

Retailers process every disputed return the same way: spreadsheet review, inconsistent criteria, growing backlog. Meanwhile **29.9%** of returns in our dataset are *not* genuine — and each needs a different response.

- **Return fraud** — deceptive claims → needs a Fraud & Risk response
- **Policy abuse** — exploiting lenient terms → needs a Policy response
- **Product quality** — genuine defects → needs a QA/Vendor response

29.9%

of returns are non-legitimate

\$4.64M

refund \$ requested by those returns — 43.8% of all refund \$

DATA OVERVIEW

60,000 orders, zero missing values, a realistic 70/12/10/8 class split



■ Legitimate — 70.1% (42,060) ■ Policy Abuser — 12.0% (7,192) ■ Fraudulent Return — 10.2% (6,112) ■ Wardrobing — 7.7% (4,636)

Behavioral signals

Photo evidence, tracking validity, address changes, refund-to-different-account, multiple accounts, dispute history

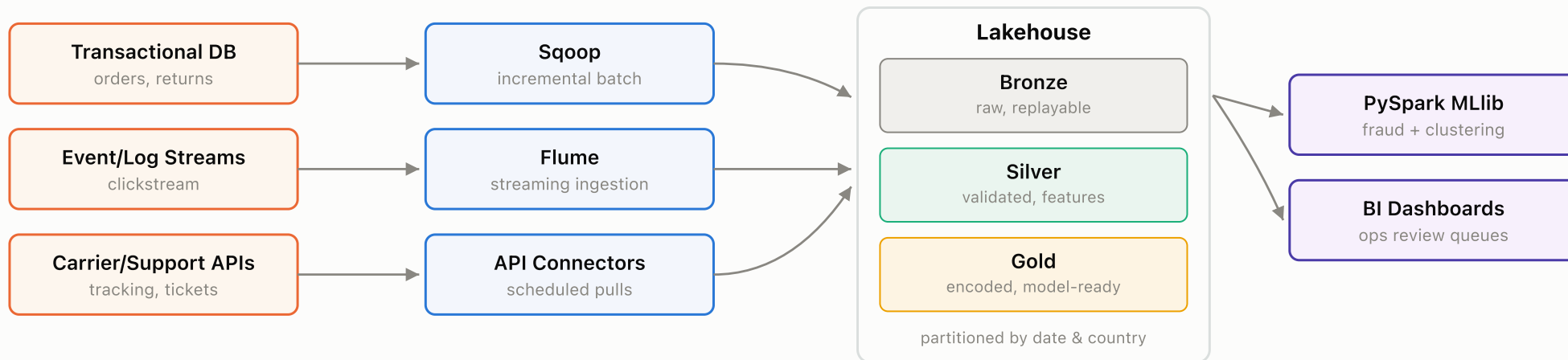
Customer history

Lifetime orders/returns, return rate, account age, loyalty segment

Context

12 product categories, 12 return reasons, 8 countries, payment method, shipping carrier

Sqoop / Flume / API ingestion → Bronze/Silver/Gold lakehouse → MLlib



Engineered features validate before they ever reach a model

Feature	Serves
fraud_signal_score	Fraud
return_to_order_ratio	Return risk
refund_to_order_value_ratio	Fraud
is_quality_issue_reason	Quality
high_value_no_evidence	Fraud

Outliers capped at 1st/99th percentile, not deleted — extreme values are often the fraud signal itself.

Sanity check that mattered

	Fraudulent Return	Legitimate
avg fraud_signal_score	2.73	0.84
avg return_to_order_ratio	0.47	0.05

Clear separation confirmed *before* trusting these features for modeling — not assumed.

Random Forest vs. Logistic Regression on return-fraud type

Model	Weighted Precision	Weighted Recall	Weighted F1
Logistic Regression (baseline)	0.998	0.998	0.998
Random Forest (final)	0.999	0.999	0.999

Class-weighted training (inverse frequency) handles the 70/12/10/8 imbalance. GBTClassifier isn't usable — Spark MLlib supports binary GBT only; Random Forest natively handles 4 classes.

Honest read: 0.999 F1 is unrealistically high — this is a Kaggle synthetic dataset with a near-deterministic labeling rule. We treat this as proof of pipeline & methodology; a realistic production target is **80–90% weighted F1**.

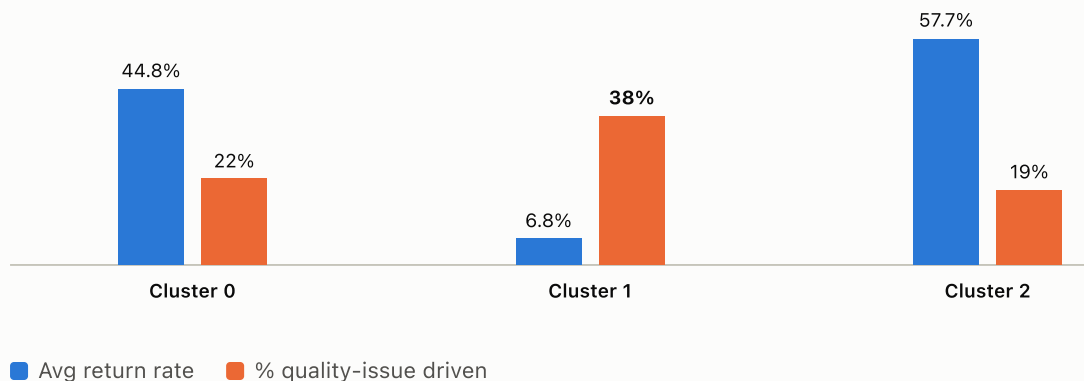
Confusion matrix (Random Forest, test set, row %)

	Legitimate	Policy Abuser	Fraud. Return	Wardrobing
Legitimate	100.0%	0%	0%	0%
Policy Abuser	0%	99.6%	0.2%	0.2%
Fraud. Return	0%	0.1%	99.9%	0%
Wardrobing	0%	0.1%	0%	99.9%

K-Means finds 3 behavioral segments the fraud label alone can't see

Cluster	n	Avg return rate	Avg order value	% quality-issue
0 — High-value fraud-risk	6,646	44.8%	\$484	22%
1 — Core legitimate base	43,829	6.8%	\$157	38%
2 — Frequent policy-abuse	9,525	57.7%	\$129	19%

Key insight: the largest, almost entirely legitimate segment still carries the *highest* quality-issue-return rate — a QA/vendor problem hiding inside "normal" customers, invisible to a fraud-only view.



RECOMMENDATIONS

Three interventions, three owners — not one blanket policy

Ship this first, no ML required: require photo evidence above a value threshold — the "high-value + no evidence" pattern alone touches **\$1.88M** of non-legitimate refund exposure (4,673 orders) and can go live before the classifier does.

Fraud & Risk

Route the highest fraud-score decile + Cluster 0 to manual review. Deploy as a triage score, not auto-block. Re-threshold asymmetrically — a false flag costs more than a slow review.

QA / Vendor Mgmt

Own Cluster 1's 38% quality-issue rate. Cross-reference dominant product category × return reason to target vendor escalations.

Customer Policy

Tighten return windows / restocking fees for Cluster 2's frequent-low-value pattern — more precise than a fraud investigation.

Before go-live: re-validate on real labeled outcomes (not this synthetic set), add a time-based holdout to catch drift, and build the human-in-the-loop labeling loop from the review queue.

What has to be true before this goes to production

Risks

- Synthetic-data metrics won't hold on live traffic — treat as a pipeline proof-of-concept, not a performance guarantee
- Real retailers rarely have clean fraud labels — needs a labeling process
- No temporal holdout here — drift is unmeasured, must be built into the rollout

Roadmap

- Real-time scoring at return-request time, not batch
- Feedback loop from reviewer overrides back into re-training
- Add unstructured support-ticket text once Flume/API ingestion is live

Thank You

Return Risk Intelligence — questions welcome.

Full detail: consulting report, `notebooks/01_data_engineering.ipynb`, `notebooks/02_ml_modeling.ipynb`, and `docs/ results write-ups`.